Cognitive Memory Manager

Persistent reasoning memory for AI coding agents

From session transcripts to durable team knowledge

Sazan Khalid & Amit Arora · Applied ML Conference 2026 · github.com/sazandkhalid/cmm

The Same Errors, Twice

The MCP Gateway Registry: 576 commits, 500+ stars. Developer A spent 43 Claude Code sessions building it. Then Developer B was assigned to the same project.

1. **Developer A's sessions**

   43 sessions over months. Auth issues debugged, deployment traps discovered, architectural decisions made and revised.

2. **Developer B's sessions**

   Same errors Developer A had already solved. Dockerfile PORT expansion failures, passlib/bcrypt incompatibility, stale module-level constants. Hours rediscovering what was already known.

3. **The question**

   Developer A's agent learned all of this. Developer B's started from zero. Why couldn't it start with that experience?

2× the debugging

Same project. Same errors. Two developers solving them independently.

INTRODUCING

Cognitive Memory Manager - CMM

Session-agnostic, model-agnostic memory management for AI coding assistants

What CMM Does

01

Capture

Extract reasoning from session transcripts: not just what the agent concluded, but the hypotheses it formed, approaches it abandoned, and pivots it took

02

Consolidate

Cluster extracted nodes and classify them into architectural insights, ranked pitfalls with resolutions, and reusable diagnostic strategies

03

Share

Sync knowledge to a team store so every future session — including a new developer's first — starts with the team's accumulated experience

```
# The full lifecycle in four commands $ cmm ingest sessions/*.jsonl -p mcp-gateway # extract reasoning nodes $ cmm consolidate -p mcp-gateway # build cognitive profile $ cmm sync ~/project # cache profile locally $ claude # agent loads profile at session start
```

PART 2

Current Landscape

Where memory stands today across platforms, frameworks, and community approaches

The Memory Landscape

Platform-Native — Claude Code Auto Memory + Auto Dream, Cursor memories. Agent-maintained notes, single developer, single machine.

Community Structured — Topic directories, CLAUDE.md rules, skill promotion lifecycles. Better organization, still single developer.

Memory Frameworks — mem0, Letta, Cognee. General-purpose agent memory with graphs, vectors, and multi-user support.

Infrastructure — Claude Managed Agents (Apr 2026), Chroma Cloud. Memory is an emerging built-in capability.

WHAT CMM ADDS

1. **Reasoning extraction** — agents stop repeating mistakes from previous sessions. The reasoning behind past decisions is preserved, not just the outcomes.
2. **Cross-developer sharing** — a new developer on the project starts with the entire team's accumulated experience instead of starting from zero.
3. **Two-scope classification** — lessons that apply everywhere compound across every project. Knowledge that only matters locally stays contained.
4. **Automated extraction** — no manual documentation or curation. Knowledge accumulates passively as developers work.

PART 3

Architecture

How reasoning is modeled, extracted, consolidated, stored, and delivered

How Reasoning Is Modeled

Each session is converted into a **Reasoning DAG** — a directed acyclic graph of typed nodes connected by causal edges. The extraction looks for specific reasoning patterns in the transcript:

Node Types

- **HYPOTHESIS** "Maybe the issue is..."
- **INVESTIGATION** Reading files, running commands
- **DISCOVERY** "Found that X is actually..."
- **PIVOT** "Different approach needed"
- **DEAD_END** Error, failed approach
- **SOLUTION** Working resolution

Each Node Stores

```
{ "node_id": "node-002-05", "node_type": "DEAD_END", "summary": "passlib[bcrypt] broken against modern bcrypt — replaced with direct bcrypt calls", "evidence": "bcrypt.hashpw() raises TypeError with passlib wrapper", "confidence": 0.88, "message_range": [45, 52], "scope": "team" }
```

Edges connect nodes causally: `DEAD_END → PIVOT` becomes `triggered_pivot`. The graph moves forward through time — no cycles.

Extraction: Two Tiers

Warm Tier — Heuristic

Three independent strategies run in parallel:

1. Keyword classification: 3–7 regex patterns per node type, confidence scales with match count (0.35 + matches × 0.1)

2. Error-resolution pairs: error message within 8 messages of a fix → paired as DEAD_END + SOLUTION at 0.60

3. Explicit conclusions: "I found that...", "The issue was..." at 0.65

When two strategies flag the same range, the higher confidence wins with a +0.05 corroboration boost.

Runs at session end via stop hook • < 1 second • Free

Cold Tier — LLM

Uses Anthropic SDK `client.messages.count_tokens()` to measure each message, then packs windows to 45% of the 200K context (≈85K tokens per window).

Each LLM call returns a **list of 1–12 nodes** classified from the window. Adjacent windows overlap by 750 tokens to prevent splitting reasoning at boundaries.

Overlap-aware dedup merges same-type nodes with overlapping message ranges, keeping the higher-confidence version.

On-demand • Confidence 0.50–0.95 • LLM API cost

```
# Warm: automatic at session end $ cmm hook stop ~/project # wired into Claude Code settings.json # Cold: upgrade warm nodes with LLM $ cmm consolidate -p mcp-gateway --upgrade # finds warm-* nodes, re-extracts with LLM
```

Building the Cognitive Profile

1 Embed
Nodes → 1536-dim vectors

2 Cluster
Cosine distance threshold 0.4

3 Classify
LLM labels type + scope

4 Rank
Frequency × severity

5 Write
cached_profile.md + ChromaDB

OUTPUT: INSIGHTS

INSIGHT Authentication is handled at the gateway layer; individual MCP servers receive pre-validated requests.

OUTPUT: PITFALLS

HIGH The healthcheck returns 200 even when backends are unreachable.

OUTPUT: STRATEGIES

STRATEGY When tool calls return empty: check adapter registration, verify MCP server, check tool name uniqueness.

```
$ cmm consolidate -p mcp-gateway # build profile $ cmm sync
~/mcp-gateway-registry # cache to project
```

Storage & Retrieval

STORE

Each node in ChromaDB

```
{ "summary": "get_iam_manager() lacks
Auth0 case...", "embedding": [0.023,
-0.041, ...1536 dims], "metadata": {
"project_id": "mcp-gateway", "node_type":
"DEAD_END", "confidence": 0.88, "scope":
"team", "developer": "alice" } }
```

Text + vector + metadata stored together. Local instance for coding, shared instance for team sync.

→

SEARCH

Retrieval

Semantic — query embedded, cosine similarity. Finds *"IAM routing"* when stored as *"get_iam_manager factory"*

```
$ cmm search "OAuth token refresh" \ --
strategy hybrid \ -p mcp-gateway
```

→

DELIVER

Two delivery paths

Session start
CLAUDE.md loads `cached_profile.md` — full profile, no API call

Mid-session
CLI skills query ChromaDB on demand

```
/search-memory "Keycloak auth" /pitfalls
/diagnose "gateway 500" /cognitive-profile
```

Getting Started

INSTALL

1. Clone & install

```
git clone github.com/sazandkhalid/cmm cd cmm python3 -m venv .venv source .venv/bin/activate uv pip install -e ".[dev]"
```

2. Get API keys

```
# Required platform.openai.com/api-keys # Optional (cold tier) console.anthropic.com/settings/keys
```

3. Verify

```
cmm --help # init, ingest, consolidate, # push, pull, review, serve
```

CONFIGURE

4. Set environment

```
cp .env.example .env # Edit .env: OPENAI_API_KEY=sk-... ANTHROPIC_API_KEY=sk-ant-... CMM_STORE_PATH= ~/.cognitive-memory/store
```

5. Initialize project

```
cd ~/my-project cmm init . # Creates: # .cognitive/config.json # .cognitive/cached_profile.md # Appends to: CLAUDE.md
```

INGEST SESSIONS

6. Find transcripts

```
# Sessions live here: ~/.claude/projects/ -path-to-project/ session-id.jsonl ls -lt ~/.claude/projects/*/*.jsonl
```

7. Ingest & build profile

```
cmm ingest sessions/*.jsonl \ -p my-project # Free mode (warm only): --no-llm cmm consolidate -p my-project cmm sync ~/my-project
```

USE IN CLAUDE CODE

8. Auto-load at start

```
# CLAUDE.md tells the agent: "Read .cognitive/cached_profile.md" # Agent gets insights + # pitfalls + strategies
```

9. Search mid-session

```
/search-memory "deploy fails" /pitfalls /diagnose "auth errors" /cognitive-profile /visualize-dag
```

Automatic mode: Add `{"hooks": {"Stop": [{"command": "cmm hook stop ."}]}}` to `settings.json` — warm extraction runs at every session end.

Team Mode

1 — CONNECT

Init with shared store

Connect your local CMM to a team-wide ChromaDB instance so all developers share memory.

```
cmm init . \ --shared /team/store \ --developer alice # Or with Chroma Cloud: --cloud-tenant "$TENANT" --cloud-database "cmm"
```

→

2 — PUSH

Push to staging

New local nodes go to the shared staging area. Not visible to teammates until approved.

```
cmm push \ -p my-project # Sends nodes to staging # collection — not main # Audit logged in SQLite
```

→

3 — REVIEW & PULL

Approve & distribute

Tech lead reviews staged nodes. Teammates pull approved memories into their local store.

```
# Tech lead: cmm review -p my-project # approve / reject / edit scope # Teammates: cmm pull -p my-project # merges approved nodes locally
```

Two scopes

PROJECT: specific to one repo, only flows to that repo's developers.
TEAM: applies everywhere, flows to all developers on all projects.

Why the gate?

One hallucinated node from a confused session should not become institutional knowledge. Every action audit-logged in SQLite with timestamp, reviewer, and reason.

PART 4

Evaluation

How we measure whether the memory actually helps

How We Measure Effectiveness

TOKEN EFFICIENCY

Does memory reduce the total compute spent on a task?

Session size (KB)
Total input + output tokens across the full session transcript

Assistant messages
Fewer messages = less context window consumed per task

Files written
Targeted fixes vs. over-engineered changes across unnecessary layers

```
# Measured from JSONL transcripts wc -l
baseline.jsonl wc -l assisted.jsonl
```

TIME TO SOLUTION

How quickly does the agent reach the root cause and a working fix?

File traversals
Number of `Read()` and `Bash()` tool calls before root cause identification

Layers explored
How many architectural layers (frontend, backend, tests, config) the agent searched through

Message to root cause
Which message number the agent first correctly articulates the underlying problem

```
$ python scripts/ controlled_comparison.py
\ --baseline no_mem.jsonl \ --assisted
with_mem.jsonl
```

DEVELOPER DELIGHT

How does the developer experience change with and without memory?

Confidence
Did the agent feel directed? Or did it feel like it was guessing and exploring blindly?

Intervention needed
How many times the developer had to redirect the agent or correct its approach

Trust in output
Would the developer ship the agent's changes without manual review? Fewer unnecessary files = higher trust.

PART 6

Demo: IAM Auth0 Routing Bug

Same bug, same codebase. One agent with 43 sessions of memory, one starting cold.

The Task & What Memory Knew

Task prompt (identical for both agents)

> *"When Auth0 is configured (`AUTH_PROVIDER=auth0`), the IAM page still shows Keycloak groups. Debug the IAM provider routing logic and fix it."*

What the profile contained

PITFALL • 95% CONFIDENCE The `get_iam_manager()` factory reads `AUTH_PROVIDER` from a module-level constant set at import time. It never updates. Auth0 case missing — falls through to Keycloak.

Resolution in profile:
Read `AUTH_PROVIDER` dynamically from `os.environ` at call time. Add Auth0 case.

The bug (before / after)

```
# BEFORE — broken AUTH_PROVIDER = os.environ.get( "AUTH_PROVIDER", "keycloak" ) # read ONCE at import def get_iam_manager():
provider = AUTH_PROVIDER.lower() # stale! No Auth0 case! return KeycloakIAMManager()
```

```
# AFTER — fixed def get_iam_manager(): provider = os.environ.get( "AUTH_PROVIDER", "keycloak" ).lower() # dynamic! if provider ==
"keycloak": return KeycloakIAMManager() elif provider == "auth0": return Auth0IAMManager() return KeycloakIAMManager()
```

Results: Baseline vs Memory-Assisted

Metric	No memory	With CMM
Assistant messages	**119**	**46**
Root cause at	msg 48	msg 37
Files written	**7**	**2**
Session size	1,127 KB	281 KB
Loaded profile	No	Yes (msg 2)
Layers explored	7 (incl. frontend)	2 (backend only)

61%
fewer messages

71%
fewer files

75%
smaller session

What happened

Baseline: broad search, wrong layers

8 Bash commands exploring full codebase. Read frontend files (IAMGroups.tsx, useRegistryConfig.ts) it never needed. Root cause at msg 48. Wrote fixes across 7 files including frontend.

Scope of Changes & Reasoning Timeline

Baseline — 7 files (over-engineered)

`iam_manager.py` • core

`management_routes.py` • backend

`config_routes.py` • backend

`useRegistryConfig.ts` • frontend

`IAMGroups.tsx` • frontend

`test_iam_manager.py` • tests

`test_management_routes.py` • tests

Assisted — 2 files (minimal, correct)

`iam_manager.py` • core

`management_routes.py` • backend

Reasoning timeline

BASELINE

msg 2: "Let me explore the codebase..."

msgs 3–10: 8 Bash commands, broad search

msgs 14–26: Frontend exploration (wrong layer)

msgs 28–38: Deeper exploration, still searching

msg 48: Root cause finally identified

Summary

What CMM does

- Extracts reasoning traces from session transcripts — hypotheses, dead ends, pivots, solutions — modeled as a directed acyclic graph
- Consolidates nodes into cognitive profiles via embedding, clustering, and LLM classification
- Two extraction tiers (warm heuristic + cold LLM) with confidence-based upgrade through dedup
- Token-budget windowing at 45% fill, backed by three context degradation studies
- Team sharing with push/review/pull workflow and project vs team scope classification
- Three-layer evaluation: interaction logging, session analysis, and controlled A/B comparison

43 sessions of reasoning on the MCP gateway — months of debugging, architectural decisions, dead ends resolved.

Ingested. Profiled. Loaded in **2 seconds**. The next developer never hit the same errors.

Open source · MIT-0 · github.com/sazandkhalid/cmm

193 tests · embeddings + LLM fully mocked · no API keys to run the test suite

Thank You

Cognitive Memory Manager

github.com/sazandkhalid/cmm

MCP Gateway Registry

github.com/agentic-community/mcp-gateway-registry

Sazan Khalid & Amit Arora · Applied ML Conference 2026